

Phase 05 — NGINX Deep Learning (Complete Notes)

The traffic officer standing in front of your application.

1. What is NGINX and why do companies use it?

NGINX ("engine-x") is a high-performance **web server** and **reverse proxy**. Created in 2004 to solve the **C10K problem** (serving 10,000 concurrent connections): Apache used a thread per connection (heavy); NGINX uses an **event-driven** model — a few worker processes, each handling thousands of connections via an event loop. Result: huge concurrency with tiny memory.

Companies put NGINX in front of app servers because it does the "front door" jobs better than any application framework:

1. **Web server** — serves static files (images/JS/CSS) directly, blazingly fast — don't waste Spring Boot threads on that.
2. **Reverse proxy** — receives all traffic, forwards API calls to the app.
3. **TLS termination** — handles all HTTPS/certificates in one place; the app speaks plain HTTP internally.
4. **Load balancer** — spreads traffic across app instances (Phase 10).
5. **Shield** — rate limiting, security headers, compression, caching, buffering slow clients.

2. Forward proxy vs reverse proxy

FORWARD proxy — sits in front of CLIENTS (server doesn't know real client)
[users] → [corporate proxy/VPN] → internet → server

REVERSE proxy — sits in front of SERVERS (client doesn't know real backend) ★ NGINX
client → internet → [NGINX] → app1 / app2 / app3

Analogy: reverse proxy = hotel receptionist. Guests never walk into the kitchen or offices; the receptionist takes every request, forwards it to the right department, returns the answer — and can refuse troublemakers (rate limit), speak all languages (TLS), and hand out brochures directly (static files).

3. Architecture we build in this phase

```
Internet —443→ NGINX → /api/* → Spring Boot :8080 (localhost only)
                  |
                  +— /* → static files — PostgreSQL :5432 (localhost only)
                     (/var/www/site)
Firewall: allow 80,443,22. Ports 8080/5432 NOT exposed.
```

4. Installation & config structure

```
sudo apt install nginx
systemctl status nginx      # runs as a systemd service (Phase 4!)
curl localhost              # default welcome page
```

```
/etc/nginx/
├─ nginx.conf                # main config (worker processes, includes)
├─ sites-available/          # one file per site (all your vhost configs)
├─ sites-enabled/            # symlinks → the sites actually live
├─ conf.d/                   # extra config snippets
├─ /var/log/nginx/access.log # every request
├─ /var/log/nginx/error.log  # ★ first place to look when something breaks
└─ /var/www/html/            # default web root
```

Change workflow — **always** this sequence:

```
sudo nano /etc/nginx/sites-available/myapp
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/
sudo nginx -t                # ★ TEST config syntax BEFORE applying
sudo systemctl reload nginx  # reload = zero downtime (restart drops connections)
```

5. Core config — server blocks (virtual hosts)

One NGINX, many sites — distinguished by the `Host` header (Phase 3!):

```
server {
    listen 80;
    server_name example.com www.example.com;    # matched against Host header

    root /var/www/example;                      # static files
    index index.html;

    location / {
        try_files $uri $uri/ =404;              # serve file, else dir, else 404
    }
}

server {
    listen 80;
    server_name api.example.com;                # same IP, different site!

    location / {
        proxy_pass http://127.0.0.1:8080;       # ★ REVERSE PROXY to Spring Boot
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Those `proxy_set_header` lines matter: without them, Spring Boot sees every request coming from 127.0.0.1 — the `X-Forwarded-*` headers preserve the real client IP and protocol (your access logs and rate limiting depend on it).

`location` matching order: exact `= /path` → longest prefix → regex `~`. Common pattern:

```
location /api/ { proxy_pass http://127.0.0.1:8080; ... } # API → app
location /      { root /var/www/site; try_files $uri /index.html; } # SPA frontend
```

6. HTTPS in NGINX — certbot + TLS config

Let's Encrypt gives free certs; certbot automates issuance + renewal + config:

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com
# proves domain control (HTTP challenge), gets cert, edits your server block, sets auto-renew timer
sudo certbot renew --dry-run
```

What certbot writes (understand it, don't just trust it):

```
server {
    listen 443 ssl http2;
    server_name example.com;
    ssl_certificate      /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/example.com/privkey.pem;
    ...
}
server {                                # HTTP → HTTPS redirect
    listen 80;
    server_name example.com;
    return 301 https://$host$request_uri;
}
```

7. Production hardening — the standard block

```
# security headers
add_header X-Frame-Options "SAMEORIGIN" always;           # no clickjacking iframes
add_header X-Content-Type-Options "nosniff" always;
add_header Strict-Transport-Security "max-age=31536000" always; # HSTS: browser insists on HTTPS
server_tokens off;                                         # hide NGINX version

# compression — smaller responses, faster pages
gzip on;
gzip_types text/plain text/css application/json application/javascript;
gzip_min_length 1024;

# rate limiting — protect the backend from abuse/bursts
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s; # http{} level: 10 req/s per IP
location /api/ {
    limit_req zone=api burst=20 nodelay;                    # allow bursts of 20, then 503
    proxy_pass http://127.0.0.1:8080;
}

# static file caching — browser caches assets, server barely touched
location ~* \.(jpg|png|css|js|woff2)$ {
    expires 30d;
    add_header Cache-Control "public, immutable";
}

# proxy cache (server-side) — cache backend responses in NGINX
proxy_cache_path /var/cache/nginx keys_zone=apicache:10m inactive=10m;
location /api/public/ {
    proxy_cache apicache;
    proxy_cache_valid 200 60s;
    proxy_pass http://127.0.0.1:8080;
}
```

8. Logs & the 502/504 you WILL meet

```
sudo tail -f /var/log/nginx/access.log # who's hitting you: IP, path, status, bytes, agent
sudo tail -f /var/log/nginx/error.log  # * connection refused / timeouts to upstream
```

- **502 Bad Gateway** → NGINX could not talk to Spring Boot: app crashed, wrong port, not started yet. Check: `systemctl status myapp`, `ss -tulpn | grep 8080`, error.log says `connect() failed (111: Connection refused)`.
- **504 Gateway Timeout** → app answered too slowly (`proxy_read_timeout` , default 60s). Fix the slow query or raise the timeout consciously.
- NGINX default 404 vs your app's 404 look different — learn to tell whose error page you're seeing.

9. Common mistakes

- Editing config and `reload` without `nginx -t` → typo takes the site down.
- Forgetting `proxy_set_header Host/X-Forwarded-For` → app sees wrong host & 127.0.0.1 clients.
- Exposing 8080 in the firewall "for testing" and leaving it open — bypasses all NGINX protection.
- `restart` when `reload` suffices (drops live connections).
- No `client_max_body_size` → file uploads fail with 413 (default is only 1 MB!).
- Redirect loop when the app also tries to redirect to HTTPS behind the proxy (fix: trust `X-Forwarded-Proto`).

10. Interview questions

1. What is a reverse proxy? Why put NGINX in front of Spring Boot instead of exposing 8080?
2. Forward vs reverse proxy?
3. How does NGINX serve many domains on one IP? (Host header / server_name.)
4. How does NGINX handle 10k concurrent connections where Apache struggled? (event loop vs thread-per-connection)
5. 502 vs 504 — causes and debugging steps for each.
6. What is TLS termination? What are X-Forwarded-For / X-Forwarded-Proto for?
7. reload vs restart ?
8. How does Let's Encrypt/certbot prove you own the domain?

11. LAB — full local build

```
# 1. install & explore
sudo apt install nginx && curl -I localhost

# 2. static site
sudo mkdir -p /var/www/mysite && echo "<h1>My Site</h1>" | sudo tee /var/www/mysite/index.html
# server block: listen 80; server_name mysite.local; root /var/www/mysite;
echo "127.0.0.1 mysite.local api.local" | sudo tee -a /etc/hosts # fake DNS locally
curl http://mysite.local

# 3. reverse proxy to a real backend
# run any Spring Boot app on 8080 (or: python3 -m http.server 8080)
# second server block: server_name api.local; location / { proxy_pass http://127.0.0.1:8080; }
sudo nginx -t && sudo systemctl reload nginx
curl http://api.local # NGINX → your app!

# 4. break it on purpose (★ the real learning)
# stop the backend → curl api.local → observe 502; read error.log; start backend → fixed
# add a typo to the config → nginx -t catches it → fix

# 5. watch logs while you browse
sudo tail -f /var/log/nginx/access.log
```

12. ASSIGNMENT 05 (submit to Claude)

1. Complete the lab. Paste: your two server blocks, the 502 line from error.log, and the access.log line of a successful proxied request.
2. Write from memory a production server block for `api.shop.com` : HTTPS (cert paths), HTTP→HTTPS redirect, proxy to :8080 with all forwarding headers, 10 r/s rate limit, gzip, 20 MB upload limit.
3. Explain the exact path of a request for `https://shop.com/product.png` vs `https://shop.com/api/products` through your NGINX config. Which never touches Spring Boot, and why is that valuable?
4. A user reports "sometimes the API gives 502 around 4:00 am". Systemd shows your app restarting nightly at 4:00. Explain what's happening and give two fixes of different quality.
5. Why is TLS terminated at NGINX and not in Spring Boot in most companies? Give 3 reasons.